

Benchmark Report: nbody

Five bodies, twenty thousand steps, and an interpreter between every calculation

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Repository: github.com/matanco64/HWSW_Project

A probe is falling toward Neptune, and somebody has to know where it will be in six hours.

Predicting motion means repeatedly adding up gravitational forces. This benchmark captures that computational core in miniature: the Sun and four gas giants, ten interacting pairs, and twenty thousand steps. The physics is compact; in Python, the machinery around each calculation is not. Our optimization asks how much of that machinery we can remove.

1. Overview: workload and result

The Computer Language Benchmarks Game's *nbody* uses symplectic Euler integration under Newtonian gravity. Each timed iteration evaluates energy, calls `advance(0.01, 20000)`, then evaluates energy again. The original kernel (the benchmark exactly as shipped in `pyperformance`, unmodified) imports only `pyperf` (the timing harness) and the standard library, with no I/O in the timed region; our Python optimization adds no library (built-in `compile/exec`), and Section 4 adds *native Rust*: a compiled Rust extension bound through PyO3, as opposed to Python run by the interpreter. State is a dict of five (`[x, y, z]`, `[vx, vy, vz]`, `mass`) tuples (35 Python floats in nested lists); `pairs` is a list of ten tuples aliasing those same lists, built once, so velocity updates are in-place list writes.

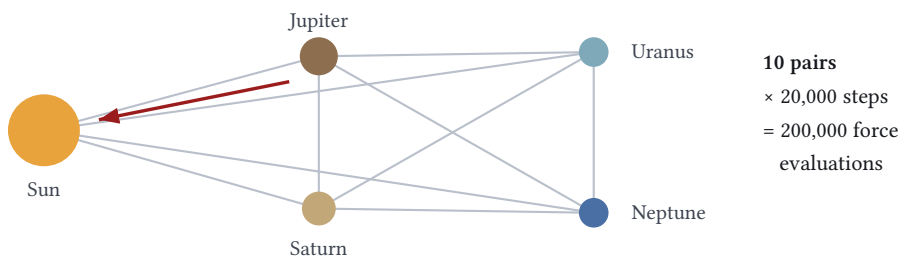


Figure 1: Ten fixed pairs × 20,000 steps = 200,000 pair-force evaluations.

The optimized Python runs 1.62× faster than the original (231.20 → 143.13 ms, a 38.1% reduction) with bit-identical state and energy; Section 4 gives the measurement, Section 2 the profile that led to it and Section 3 the change itself.

2. Initial analysis

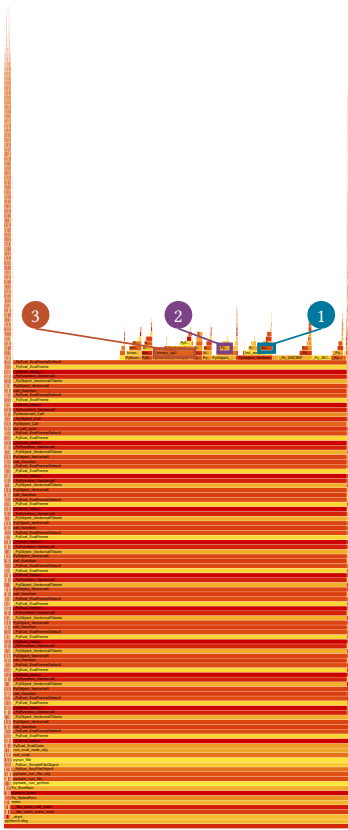
py-spy (Python frames, release CPython) places almost all benchmark work in advance(). A perf record profile (999 Hz cpu-clock, DWARF call graphs, debug build python3-dbg, rendered with FlameGraph) identifies generic arithmetic dispatch, float allocation and list access as substantial costs. These debug-build percentages locate costs to investigate; release-build timing establishes the optimization's benefit.

nbody, C-frame profile	Original self	Original incl.	Optimized self
_PyEval_EvalFrameDefault	44.10%	99.31%	40.66%
binary_op1 (generic arithmetic dispatch)	4.90%	20.07%	8.97%
float_mul	3.86%	3.86%	5.09%
PyFloat_FromDouble	4.03%	5.82%	5.62%
float_dealloc	3.48%	3.48%	4.37%
list_ass_item	1.78%	1.78%	–
PyNumber_AsSsize_t	2.22%	4.87%	0.03%
Float object handling, all symbols	16.25%	–	21.62%
List access and index conversion, all symbols	11.26%	–	0.06%

Self is time in the function itself; **inclusive** is time in it and everything it calls. The two grouped rows sum self time over $^{\wedge}(\text{float_}| \text{PyFloat_})$ and over $^{\wedge}(\text{list_}| \text{listiter_}| \text{PyNumber_AsSsize_t}| \text{PyLong_AsSsize_t})$ respectively. The optimized column is the same profiler on the optimized benchmark (results/perf_report_nbody_opt.txt, results/flame_nbody_opt.svg); each column has its own denominator, so shares of the remaining time rise for the families that stay. List access and index conversion fall from 11.26% to 0.06% of self time (the indexing left the loop); float object handling does not fall (the arithmetic still allocates a Python float per operation), which is what the native tier in Section 4 removes. The figure labels give total inclusive shares across a function's recorded frames; the outline identifies one representative frame. The flat perf table and the SVG may differ slightly through rounding and inline attribution (list_ass_item: 1.78% self here, 3.40% inclusive in the graph). Appendix A4/A6 records the extraction method.

Reading the profiles

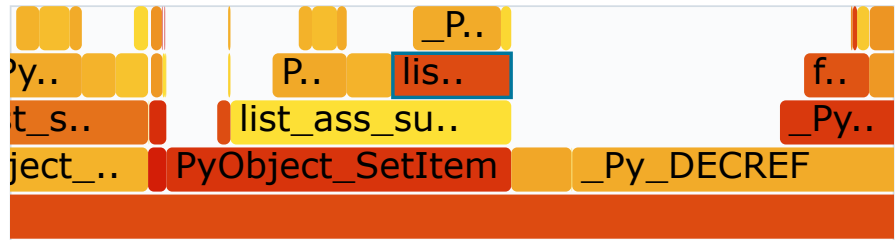
Full recorded flame graph



Highlighted regions - enlarged with surrounding frames

1. List stores

list_ass_item | 3.40% inclusive



2. Index conversion

PyNumber_AsSsize_t | 4.87% inclusive, across 2 per-line frames; widest 2.73% outlined



3. Generic arithmetic dispatch

binary_op1 | 20.09% inclusive, across 12 per-line frames; widest 10.55% outlined

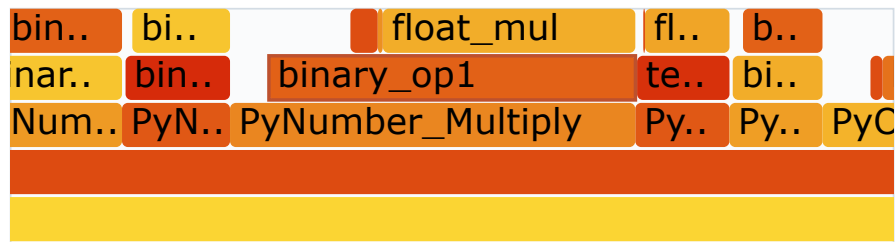
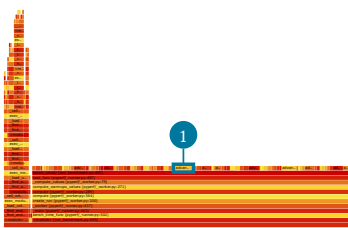


Figure 2: Full original C-frame profile (debug CPython), with the same call paths outlined and enlarged. All original frames, widths and startup context are retained. Each label gives the function's total inclusive share; the outlined box is the widest of the per-line frames the profiler splits it into, and callers sit below it.

Full recorded flame graph



Highlighted regions - enlarged with surrounding frames

1. Generated integration loop

advance | 91.57% inclusive, across 95 per-line frames; widest 4.76% outlined

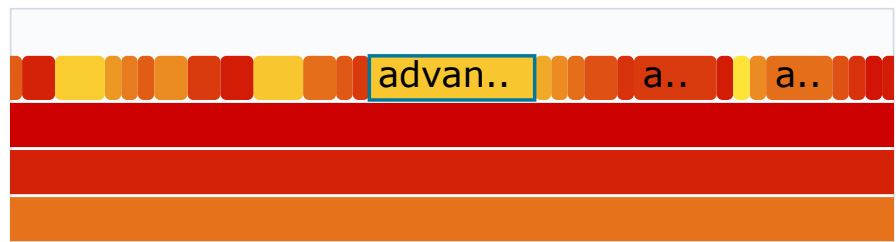


Figure 3: Full optimized Python-frame profile (210 samples) with the integration call path enlarged. Integration remains the hotspot. Independent normalization does not display the absolute runtime reduction from 231 to 143 ms.

Reading the flame graphs: the original graph is a single tower, `_PyEval_EvalFrameDefault` (99.31% inclusive) with `binary_op1` (20.07%), float allocation/free and list indexing as children; there is no second hotspot. In py-spy Python frames `advance` holds 97.7% of samples before (310 samples) and 91.4% after (210 samples); the rest is start-up and imports. Originals: `results/flame_nbody_stock_full.svg`, `results/pyspy_nbody_opt_full.svg`.

3. Optimizations

Specialize the fixed schedule

The original loop revisits the same pairs and masses on every step. At import time, the optimized version emits an `advance()` function with the pair loop unrolled, coordinates in Python locals, and masses as literals. It loads state before the step loop and writes it back afterwards, so energy reporting still observes the original body objects.

This removes list subscripting from the generated step loop and avoids repeated pair unpacking. List operations remain at function entry and exit, amortized over 20,000 steps.

Before and after: the x update of one pair

The following excerpts abbreviate the shipped transformation. The y/z operations and other pairs are omitted only from the illustration. M0 and M1 denote the numeric mass literals that the generator inserts; they are not new runtime lookups.

Original: inside the pair loop

```
for ([x1, y1, z1], v1, m1),
    ([x2, y2, z2], v2, m2) in pairs:
    dx = x1 - x2
    # dy, dz and mag as below
    b1m = m1 * mag
    b2m = m2 * mag
    v1[0] -= dx * b2m
    v2[0] += dx * b1m
```

Generated: pair (0, 1)

```
dx = x0 - x1
# dy, dz and mag as below
b1m = M0 * mag
b2m = M1 * mag
ux0 -= dx * b2m
ux1 += dx * b1m
```

Both compute $\text{mag} = \text{dt} * ((\text{dx}^2 + \text{dy}^2 + \text{dz}^2) ** (-1.5))$ in the same order. The optimization removes repeated pair unpacking and list reads/writes, not the force calculation. Coordinates and velocities enter locals once per invocation; after all steps, the generator writes them back to the original lists. These are still Python float objects, so removing indexing does not remove all interpreter arithmetic or allocation costs. Source: `benchmarks/bm_nbody/run_benchmark.py, _advance_source`.

Correctness constrains the arithmetic. The kernel retains $\text{dt} * (\text{dsq} ** -1.5)$ and the original pair-update order. A square-root/division replacement changes rounding and is not used in the shipped bit-exact version. The emitter accepts a body list and pair schedule, so the same transformation extends to larger systems.

4. Performance comparison

Original vs optimized Python

Configuration	Mean ± SD	vs original
Original Python	231.20 ± 7.85 ms	1.00×
Optimized Python	143.13 ± 1.56 ms	1.62×

The optimization reduces runtime by **38.1%**, exceeding the course's 7% requirement. It specializes the interaction schedule into local-variable arithmetic while preserving operation order. Final state and energy compare exactly equal to the original after 20,000 steps. Energy is a useful physical sanity check; componentwise state comparison is the stronger equivalence check.

Measurement scope: Course QEMU/KVM virtual machine (VM), Ubuntu 22.04, release CPython 3.10.12, pyperformance 1.14.0 with pyperf 2.10.0; 120 measured values per configuration (40 value-bearing worker runs × 3 values, calibration separate), every timed run pinned to guest CPU 0, measured from revision 2c8c754 through the documented runner scripts. Source: `results/vm_canonical_20260910_2c8c754/suite/{baseline,optimized}_nbody.json`, which also record the backend and CPU affinity (Appendix A7). SD denotes sample standard deviation, not a confidence interval. A later commit removed a dead store from the generated `advance()` and widened the optional-import fallback; both pass the same bit-exactness oracle but were not re-timed. Profiling is separate from timing; the shared appendix records methods and provenance.

Worker processes account for much of the timing variation. Appendix A5 reports medians, interquartile ranges and uncertainty adjusted for worker clustering; the 88.1 ms reduction is about 70 times the combined clustered standard error (1.24 and 0.24 ms).

Native tier: native execution does less work, not higher IPC

The Rust/PyO3 System executes all 20,000 steps in one call, with both energy evaluations also native. State and energy match the original exactly on the tested VM build, and not only at five bodies: the same check at 5, 10 and 31 bodies is bit-identical in state and energy after 20,000 steps (results/vm_verify_20260921_f407567/check_all.log). That matters because the pair schedule above five bodies is generated code the default workload never exercises. Exactness is a property of this build; it does not guarantee equality across all compilers and math libraries.

Same benchmark file	Mean $\pm$ SD	vs Python
Optimized Python (fallback backend)	145.30 $\pm$ 4.97 ms	1.00×
Native Rust backend	9.530 $\pm$ 0.050 ms	15.25×

Source: canonical suite/{fallback,native}_nbody.json (Appendix A7). Both use direct pyperf; the original-vs-optimized comparison above uses pyperformance. Against the canonical original, native is 24.26× faster. Different Python denominators prevent multiplying the two reported ratios. Explicit backend selection and worker metadata establish what ran; native mode fails if unavailable.

Matched-work CPU counters

A separate pinned warm-loop experiment repeats the work 64 times per run. Medians of three runs are normalized per iteration, excluding setup/warmup (Appendix A2). Rates are medians of per-run rates, so they can differ from the ratio of the displayed medians (cache: 59.6/3,006 = 1.98%). All events are user-mode.

Metric per iteration	Optimized Python	Native Rust
Elapsed time	141.08 ms	9.462 ms
Instructions	1,103.62 M	41.32 M
Cycles	336.24 M	22.55 M
IPC	3.28	1.83
Branches	185.97 M	3.92 M
Branch misses (rate)	583,560 (0.314%)	60 (0.00153%)
Generic cache references (perf cache - references)	3,006	255
Generic cache misses (rate)	59.6 (2.245%)	2.5 (1.134%)

Native needs 26.7× fewer instructions and 14.9× fewer cycles. Removing Python dispatch and object handling reduces work. Lower IPC (instructions per cycle) still yields faster execution because so many fewer instructions remain; IPC alone does not measure useful work.

Cache misses are sparse and variable (Python 59–94, native 2–10 per iteration across three runs); they do not establish a cache bottleneck. Invalid L1 load counts are omitted.

5. Beyond the benchmark: what changes when N grows?

At five bodies, Barnes-Hut (BH, an $O(N \log N)$ tree approximation) is **3.92× slower**: tree setup/traversal does not pay off for ten pairs. Development struct-of-arrays and NumPy variants also lose at this size (0.88× and 0.35–0.46× of the original’s speed respectively); the fast multipole method (FMM) was not measured. Unrolled source grows as $O(N^2)$, so the shipped generator falls back to the ordinary (rolled) pair loop above 20,000 pairs. Detailed code-size experiments are WSL2/CPython 3.10.21 development evidence, distinct from the VM sweep below (dev/nbody/FINDINGS.md).

The pure-Python VM force-evaluation sweep below includes a rebuilt Barnes-Hut tree at each evaluation ($\theta = 0.5$, best of seven; ratios use unrounded times). It evaluates an extended workload, not the required $N = 5$ benchmark.

N	Direct	BH	BH/direct	Median rel. error
5	0.020 ms	0.077 ms	3.92×	0
100	4.746 ms	8.410 ms	1.77×	1.6e-5
300	42.977 ms	42.489 ms	0.99×	3.3e-5
800	313.489 ms	158.846 ms	0.51×	8.7e-5
1,600	1,254.660 ms	382.471 ms	0.30×	1.8e-4

The crossover is near $N = 300$ for this distribution (near-coplanar circular orbits added outward from 40 AU); median acceleration error is not a worst-case or trajectory bound. A separate VM native sweep gives 21–22× over rolled Python at $N = 100$ –3,200, using approximately equal pair-update counts (results/bigN_sweep.txt).

6. Hardware acceleration proposal: current design

`grape_pipeline` is our accelerator for the `advance()` kernel. The name follows GRAPE (“GRAVity Pipe”), the University of Tokyo family of special-purpose N-body machines, which hard-wire the pairwise-force formula as a pipeline and leave the rest to a host. Ours differs in two ways that the benchmark forces. It computes in full IEEE-754 FP64 in the benchmark’s own operation order, because reduced-precision pair forces diverge from the Python energy trace. It also retains state on the device and executes a complete `advance(dt, n)` request, rather than returning forces to a host integrator. Force computation suits a scheduled datapath, while velocity accumulation must respect dependencies between pairs sharing a body. Steps remain serial.

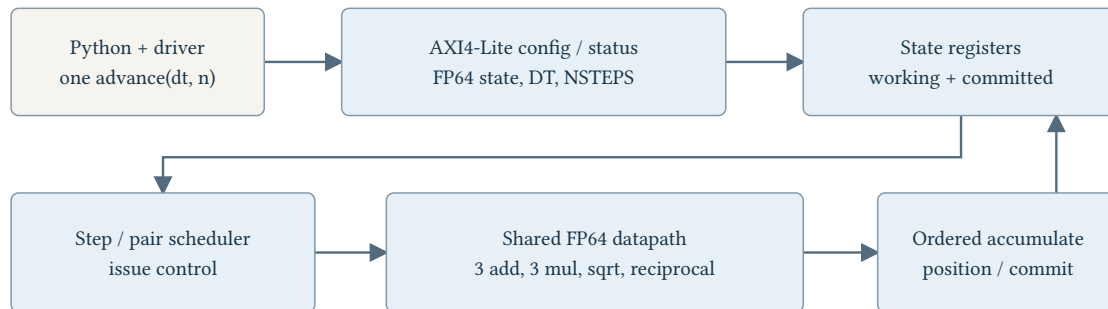


Figure 4: Block diagram: current datapath and software boundary, summarized from the approved architecture. Multiplication and accumulation round separately; there is no FMA unit.

Block diagram in words: Python driver → AXI4-Lite registers (FP64 state, DT, NSTEPS, pair list) → step/pair scheduler → shared FP64 units (3 add, 3 multiply, square root, reciprocal) → ordered accumulate → working and committed state registers → read back over AXI4-Lite when STATUS.DONE or `irq` is raised. All traffic is register access; there is no stream or DMA port. Full drawing: `hw/grape_pipeline/docs/block_diagram.svg`.

Datapath: Three FP64 add/subtract units, three multipliers, one square-root unit and one reciprocal unit share a scheduled 290-operation step. The square root uses radix-4 iteration; the reciprocal uses a ROM seed and refined fixed-point arithmetic. An ordered accumulation sequencer resolves body-component dependencies before positions are updated and committed.

Interface: A 32-bit AXI4-Lite slave exposes FP64 body state and DT, 32-bit NSTEPS, pair configuration, control/status, counters and IRQ. Software loads state and parameters, starts one request, waits, then reads state. There is no per-step DMA or Python call. A register-level driver model implements this protocol, is checked against the register map, and is co-simulated against the RTL through the same AXI4-Lite agent the testbench uses (`hw/grape_pipeline/tb/test_driver_model.py`): one `advance(0.01, 2)` invocation over the register protocol reads back all 35 state components bit-identical to the golden model in 250 cycles. It is not a deployed device driver or evidence of a physical CPU/accelerator run.

Generality: the RTL is generated for five bodies (`grape_regs.sv`: the body window is $N_{\text{BODIES}} \times 7$ FP64 words held in flops; the pair list holds at most ten entries; the 290-operation step’s issue schedule is a 200-event ROM emitted by `docs/gen_reservation.py` for that pair list (`rtl/grape_sched_rom.svh`), the remaining accumulate and integrate operations being ordered dynamically). A larger N keeps the datapath, the interface (`advance(dt, n)` over AXI4-Lite) and the ordered accumulation, but moves body state from flops to SRAM, regenerates the schedule, and, as the model below shows, wants more arithmetic units. The brief’s preference for hardware that is not too workload-specific is met at the level of the operation (pairwise force and ordered accumulate, the GRAPE recipe), not of this five-body instance.

Dependencies determine the schedule

Pairs (0, 1) and (0, 2) can form their distance terms from the same step’s positions, but both update body 0’s velocity. Its x lane must compute $u0a = u0 - dx01*b1m$, then $u0b = u0a - dx02*b2m$; issuing both against the old $u0$ would lose an update. Regrouping them into one subtraction may also change FP64 rounding. Different body-component lanes can progress independently; a dependency tracker and ordered issue logic

wait for each lane’s preceding result. Positions update only after their required velocity contributions, and the next step uses the committed positions.

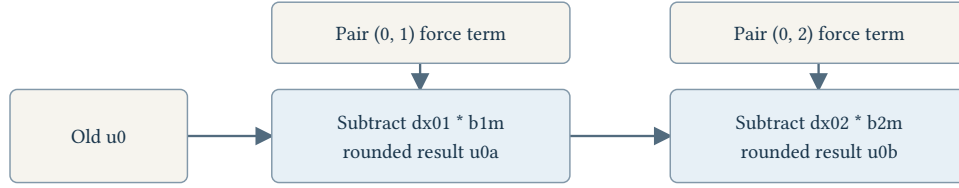


Figure 5: Illustrative dependency chain for body 0’s x velocity. Force terms may be prepared independently, while the two rounded velocity updates remain ordered.

Verification and physical cost

How these numbers were produced: the accelerator is written in SystemVerilog and simulated cycle by cycle (Verilator, cross-checked with Icarus) inside a Python testbench that runs the real 20,000-step benchmark and compares every result against a *golden model*, a Python reference of the same arithmetic. Cycle counts, test results and coverage come from these runs. The design is then *synthesized*: Yosys translates it into a netlist of standard logic cells from *sky130*, the open-source SkyWater 130 nm process, which gives cell count and area. Finally OpenLane *places* the cells, builds the clock tree and runs *static timing analysis*, which sums gate and wire delays along every register-to-register path; the slowest path sets the maximum clock. We stop after clock-tree synthesis (post-CTS): cells and the clock network are placed but signal wires are not routed, so the frequency is an estimate, not silicon. The toolchain is entirely open source, so every number can be regenerated from the repository.

Quantity	grape_pipeline	Produced by
Cycles per step	124	RTL simulation against the golden model
Full-benchmark cycles	2,480,000; 0 mismatches	same, 20,000 steps
Directed + random tests	9 of 9	same, Verilator and Icarus
Line / toggle coverage	91.7% / 96.0%	same
Cells	584,454	Yosys synthesis onto sky130 cells
Area	4.075 mm ²	same
Timing-derived frequency	≈ 19.46 MHz	OpenLane: place, clock tree, static timing
Power estimate	≈ 19 mW	same, default switching activity

Evidence levels: the schedule model predicted 123–127 cycles/step and RTL measured 124. The 19.46 MHz estimate derives from a 150 ns constraint and +98.6212 ns worst setup slack at the typical corner; the critical path is the integrate-multiplier operand path. It is a preliminary timing estimate rather than measured silicon performance: there is no routed sign-off or final layout (GDS), and the design has not demonstrated its 50 MHz target. The power figure is a tool estimate at the run’s 150 ns clock constraint, not workload power. The table’s cell count and area come from a plain Yosys synthesis made before the final rewrite of the issue-selection logic into balanced trees. The OpenLane synthesis that produced the timing maps the final RTL to 446,932 cells and 4.66 mm² (5.65 mm² after buffering and the clock tree, 39% of the core). The two recipes are not comparable with each other; within the OpenLane recipe the rewrite made the design 5% smaller (4.92 to 4.66 mm²) as well as faster (hw/grape_pipeline/synth/evidence/area_openlane.txt).

Recorded architecture	Mapped cell area	Cycles/step
One-wide accumulation, one-port register file	2.938 mm ²	162
Three-wide accumulation, three-port register file	4.075 mm ²	124

The three-wide design spends 38.7% more mapped area to reduce cycles/step by 23.5%. More parallelism can reduce cycles while adding selection, forwarding and wiring cost that limits the clock.

Does the hardware win?

How the end-to-end time is estimated: offloading a stage removes its share of the software time and adds the hardware’s time plus the cost of moving data, $T_{\text{new}} = (1 - f) \cdot T_{\text{sw}} + T_{\text{hw}} + T_{\text{if}}$. T_{sw} is the measured software run time and f the fraction of it that moves to hardware. T_{hw} is RTL cycles divided by

the clock frequency. T_{if} is the interface time, here about 150 AXI-Lite register transactions per run, which is not measured. We assume 5% of the original runtime (harness and energy evaluation) stays in Python; that residual is an assumption, not a matched phase measurement (the measurement is one pyperv run of the harness with advance stubbed out, and it is the next thing to measure).

Tier	Time / run	vs original	Evidence
Original Python	231.20 ms	1.00×	measured
Optimized Python	143.13 ms	1.62×	measured
Native Rust	9.53 ms	24.26×	measured
Hardware at 19.46 MHz (achievable)	≈ 139 ms	≈ 1.66×	projected: 127.4 ms compute + residual
Hardware at 50 MHz (design target)	≈ 61 ms	≈ 3.8×	projected: 49.6 ms compute + residual

Verdict: at the achievable clock the accelerator is a projected 1.66× over the original, level with the optimized Python tier (1.03×) and about 15× slower than the native Rust tier; it would still be about 6× slower at the 50 MHz target. One step costs 124 cycles, 6.4 μs at 19.46 MHz, against 0.48 μs natively. The benchmark’s cost was interpreter overhead rather than arithmetic, so removing the interpreter captures almost all of the gain, and at $N = 5$ with ordered pair dependencies there is little parallelism for custom FP64 hardware to exploit (the next subsection projects larger N). Matching Rust would need the datapath alone to run at about 260 MHz, and the assumed 11.6 ms Python residual already exceeds Rust’s 9.53 ms on its own. These are projections: no run of Python attached to hardware exists, and no energy-efficiency advantage follows without workload power and complete system measurements.

Would a larger N change the verdict? (model projection)

Section 5 measured software at larger N ; the RTL is fixed at five bodies, so the hardware side can only be projected. We ran the same schedule model that predicted the RTL (123 cycles/step modeled, 124 measured) with N bodies instead of five, keeping every unit latency (hw/grape_pipeline/docs/schedule_model_n.py). At $N = 5$ a step is *latency-bound*: one pair’s chain through subtract, square, square root, reciprocal and accumulate is about 80 cycles, and ten pairs cannot fill it, so the step costs 12.3 cycles per pair in the model (12.4 measured). With more pairs the pipelined units stay busy and the cost falls to the unit-bound limit of about 4.4 cycles per pair (eleven multiplies and eleven adds per pair over three units of each). The ordered-accumulation constraint also fades: $3N$ independent velocity lanes interleave, so no lane waits on itself.

Datapath (model, $N = 100$)	Cycles/pair	μs/pair	vs native
As built: 3 add, 3 mul, 1 sqrt, 1 rcp	4.36	0.224	5.0× slower
Wider: 12 add, 12 mul	1.10	0.056	1.25× slower
Widest: 24 add, 24 mul, 2 sqrt, 2 rcp	0.56	0.029	1.6× faster

Times are at the 19.46 MHz clock; rcp is the reciprocal unit; the two wider rows also let square root and reciprocal accept a new pair every cycle (as built: every second cycle). The software references are Section 5’s VM sweep: 0.98 μs per pair-update for rolled Python and 0.045 μs native. The as-built datapath would therefore be about 4.4× faster than Python at $N = 100$ (2.8× better per pair than at $N = 5$) but still 5× slower than one native core; at the 50 MHz target that gap is 1.9×. Passing native software needs roughly four to eight times the arithmetic units, which is GRAPE’s own recipe: replicate the pair pipeline. These figures are upper bounds for the hardware. They assume the clock survives the wider issue-selection logic, while our one-wide versus three-wide measurement shows the opposite trend; body state must move from flops to SRAM; and above $N \approx 300$ the software competitor becomes Barnes-Hut rather than direct summation. $N = 5$ is the worst case for this architecture, not a representative one.

The hardware’s square-root/reciprocal expression differs from original pow. RTL is checked against its own arithmetic model. Comparison with the original targets relative energy error $\leq 1e-12$ and per-body position/velocity errors $\leq 2e-9 / 5e-11$ at completion. The full 20,000-step RTL run meets them: relative energy error $1.7e-14$ and position/velocity errors $2.1e-12$. These tolerances are distinct from the software tier’s observed exact equality.

7. Conclusion

Specializing a fixed schedule cuts Python runtime by 38.1% while preserving the tested state and energy exactly. Native execution removes far more interpreter work through the same coarse advance(dt , n) interface and reaches 9.53 ms, 24.26 $\times$ over the original.

In hardware, `grape_pipeline` executes the whole `advance()` kernel over that same boundary, bit-exactly against its arithmetic model, in 124 cycles per step. At the 19.46 MHz clock that static timing supports this projects to about 139 ms per run: 1.66 $\times$ over the original, level with the optimized Python, and about 15 $\times$ slower than the native tier (about 6 $\times$ at the 50 MHz target), for about 4.7 mm² of 130 nm standard cells at synthesis (4.1 mm² under plain Yosys before the last rewrite). The benchmark's cost was interpreter overhead rather than arithmetic, so removing the interpreter captures nearly all of the gain. At five bodies with ordered pair dependencies there is too little parallelism for custom FP64 hardware to repay its area: a step is latency-bound at 12.3 cycles per pair in the model (12.4 measured), where the schedule model projects 4.4 at $N = 100$ and native-class throughput only with four to eight times the arithmetic units. The measured cycle/area trade-off shows why a fast schedule alone is insufficient: the parallelism that saves cycles also lengthens the clock path.

Evidence: `hw/grape_pipeline/docs/uarch.md`, `prd.md`, `testplan.md`, `ppa.md` and `integration.md`; `hw/docs/hardware_report.md`. Reproduction and supporting profiles: **report_appendix.pdf**.